CS208 Lab Assignment 2: File Compression & Decompression using Huffman Coding

> Deadline: May.6/7/8. Late submission is not allowed
>
> Verification Method: In-person demonstration during lab sessions.

Objective

In this assignment, you will implement a complete **file compression and decompression system** based on **Huffman Coding**.

By the end of this assignment, you will:

- Understand and implement Huffman Coding
- Design a custom file format
- Bit-level file I/O operations for binary data processing
- Build independent compressor and decompressor executables
- Verify correctness using automated scripts

Task Overview

1. Compressed File Format Design

Your compressed file must follow this structure:

```
+----------------------------+
| Header                     |
|  - original file size      |
|  - Huffman Tree structure  |
+----------------------------+
| Encoded Data (bitstream)   |
+----------------------------+
```

Your header must include:

- Original file size: 4-byte unsigned integer (used to truncate padded bits during decompression)
- Information to reconstruct Huffman Tree: Serialized tree structure (must allow full reconstruction of the tree)
- Padding handling: padding bits (0-7) added to the end of the bitstream to align with bytes

You must clearly describe your format in `README.md`, including:

- Data type and byte length of each header field
- Huffman tree serialization/deserialization rules
- Padding bit calculation and processing logic

2. Compressor (Encoder)

Your compressor must:

- Read input file (supports all binary/text file types)
- Count the frequency of every byte in the input file
- Build Huffman Tree using a min-heap/priority queue
- Generate unique Huffman codes for each byte
- Encode input data into **encoded bitstream**
- Write **file header** + **encoded bitstream** to output file
- Correctly handle padding bits for byte alignment

3. Decompressor (Decoder)

You must implement a **separate executable program** that:

- Must NOT **depend on compressor code at runtime**
- Read compressed file
- Read original file size

- Reconstruct the Huffman Tree from the file header
- Decode the bitstream back to the original bytes using the Huffman tree
- Ignore padding bits and output the decoded file

4. Automated Verification Script

Required Project Structure

```
huffman/
│
├── src/                    # Source code
│   ├── compressor/         # Compression-related modules
│   ├── decompressor/       # Decompression-related modules
│   └── common/             # Shared utilities
│
├── bin/                    # Compiled executables
│   ├── compress            # Compressor executable
│   └── decompress          # Decompressor executable
│
├── testcases/              # Test files
├── output/                 # Generated files
├── README.md               # Project instructions
└── run_verify.sh / run_verify.bat / run_verify.py   # Build & run script & compare original and
```

Automated Verification Script

Write a runnable script (Bash/Python recommended, named run_verify.sh/run_verify.bat/run_verify.py) that automatically completes the full pipeline:

1. Clean the `bin/` and `output/` directory (ensure it is empty before execution)
2. Compile the `src/` , putput the executables to `bin/`
3. Traverse all files in `testcases/` , Call the `compress` executable to compress files and save `.huff` files to `output/`
4. Call the `decompress` executable to decompress `.huff` files in `output/` and save `decoded_*` files to `output/`
5. Byte-by-byte comparison between the original file and the decoded file, output clear results: `[Pass] File XXX vs decoded_XXX` or `[Fail] File XXX vs decoded_XXX`

Sample output of script execution:

```
===== Step 1: Clean =====
Clean finished

===== Step 2: Compile =====
Compile finished

===== Step 3: Compress =====
Compressing testcases\empty.txt

Compressing testcases\large.txt

Compressing testcases\random.bin

Compressing testcases\small.txt

Compressing testcases\sustech.jpeg

===== Step 4: Decompress =====
Decompressing output\empty.txt.huff to output\decoded_empty.txt

Decompressing output\large.txt.huff to output\decoded_large.txt

Decompressing output\random.bin.huff to output\decoded_random.bin

Decompressing output\small.txt.huff to output\decoded_small.txt

Decompressing output\sustech.jpeg.huff to output\decoded_sustech.jpeg

===== Step 5: Verification =====
[Pass] empty.txt vs decoded_empty.txt
[Pass] large.txt vs decoded_large.txt
[Pass] random.bin vs decoded_random.bin
[Fail] small.txt vs decoded_small.txt
[Pass] sustech.jpeg vs decoded_sustech.jpeg
===== DONE =====
```

In-Person Verification & Demonstration Workflow

Step	Action	Expected Outcome
1	Check directory structure	Matches required layout exactly
2	Clean environment test	Delete `bin/` and `output/`, then run Verification script
3	Script execution	Script runs without errors, compiles code, and prints PASS/FAIL for all testcases
4	Output inspection	`output/` contains `.huff` and `decoded_*` files
5	Compression ratio check	Text files show significant size reduction; binary/image files may increase slightly
6	Visual/Functional check	Open a decoded image/binary/text file → must be identical to the original
7	Conceptual Q&A	asks 1–2 questions (e.g., padding handling, tree serialization, why binary files don't compress well)
8	README review	Clear compilation/run instructions, example commands and format design

Grading Rubric & Deduction Policy (Total: 100 points)

Category	Points	Deduction Rules
Project Structure	5	`-5` if directory layout or executable names mismatch requirements
Automated Verification Script	10	`-10` if script fails to compile, run, or produce comparison output; `-5` if not fully automated (requires 1~2 manual steps)
Correctness & Compression	65	`-13` per failed testcase (5 testcases total). Any data loss, corruption, or mismatch → `0` for that case.

Category	Points	Deduction Rules
Documentation (`README.md`)	10	`-2` to `-10` if missing or lacks: compile/run steps, example commands, **detailed file format design**
Conceptual Q&A	10	`-5` to `-10` if unable to explain padding, tree reconstruction, or binary compression behavior

- **Plagiarism or AI-generated code without attribution:** `0` (subject to academic integrity policy)